Analysis and Design of Algorithms (ADA)

Lab Assignment – 2

Submitted by : Md. Anas khan

Submitted to : Dr. Aarti

Course : B.Tech – CSE (Data Science)

Roll no. : 2401420035

Problem Context

In real-world systems, optimization and decision-making problems arise in logistics, finance, healthcare, and networking. Different algorithmic paradigms such as Divide & Conquer, Greedy, and Dynamic Programming are used to solve these efficiently.

Learning Objectives

- Implement Divide & Conquer, Greedy, and Dynamic Programming algorithms
- Analyze performance using time and memory
- Compare scalability across algorithms
- Understand real-world applicability

Setup Instructions

```
python -m venv .venv
source .venv/bin/activate    # Linux/Mac
.venv\Scripts\activate       # Windows
pip install matplotlib numpy memory_profiler jupyterlab
jupyter lab
```

Project Structure

```
algo-strategies-mini-project-siddhant
├── README.md
├── .gitignore
├── notebooks/
│   └── algo_strategies_notebook.ipynb
├── images/
├── docs/
```

Task 1: Divide & Conquer (Merge Sort)

Code

```
import time
import random
import matplotlib.pyplot as plt

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1


    result.extend(left[i:])
    result.extend(right[j:])
    return result

sizes = [100, 500, 1000, 2000]

times = []

for size in sizes:
    arr = [random.randint(1,10000) for _ in range(size)]
    start = time.time()
    merge_sort(arr)
    end = time.time()
    times.append(end-start)

plt.plot(sizes, times)
plt.xlabel("Input Size")
plt.ylabel("Time")
plt.title("Merge Sort Performance")
plt.show()
```

Observation

- Time Complexity: O(n log n)
- Efficient and stable for large datasets

Task 2: Sorting Comparison

Code

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)

merge_times = []
quick_times = []

for size in sizes:
    arr = [random.randint(1,10000) for _ in range(size)]

    start = time.time()
    merge_sort(arr)
    merge_times.append(time.time() - start)

    start = time.time()
    quick_sort(arr)
    quick_times.append(time.time() - start)

plt.plot(sizes, merge_times, label="Merge Sort")
plt.plot(sizes, quick_times, label="Quick Sort")
plt.legend()
plt.show()
```

Observation

- Merge Sort: Stable and predictable
- Quick Sort: Faster on average but worst-case $O(n^2)$

Task 3: Greedy Algorithm (Activity Selection)

Code

```
def activity_selection(start, end):
    activities = sorted(zip(start, end), key=lambda x: x[1])

    selected = [activities[0]]

    for i in activities[1:]:
        if i[0] >= selected[-1][1]:
            selected.append(i)

    return selected

start = [1,3,0,5,8,5]
end = [2,4,6,7,9,9]

print(activity_selection(start, end))
```

Observation

- Greedy works when local optimum leads to global optimum
- Used in scheduling and resource allocation

Task 4: Dynamic Programming (Knapsack)

Code

```
def knapsack(W, wt, val, n):
    dp = [[0 for _ in range(W+1)] for _ in range(n+1)]

    for i in range(n+1):
        for w in range(W+1):
            if i == 0 or w == 0:
                dp[i][w] = 0
            elif wt[i-1] <= w:
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]],
                               dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][W]

val = [60,100,120]
wt = [10,20,30]
W = 50

print(knapsack(W, wt, val, len(val)))
```

Observation

- Time Complexity: O(nW)
- Stores overlapping subproblems for optimal solution

Task 5: Travelling Salesman Problem (TSP)

Code

```
from itertools import permutations

def tsp(graph):
    n = len(graph)
    nodes = list(range(n))
    min_path = float('inf')

    for perm in permutations(nodes[1:]):
        cost = 0
        k = 0
        for j in perm:
            cost += graph[k][j]
            k = j
        cost += graph[k][0]

        min_path = min(min_path, cost)

    return min_path

graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]

print(tsp(graph))
```

Observation

- Time Complexity: O(n!)
- Demonstrates exponential growth

Performance Profiling

```
from memory_profiler import memory_usage

def test_memory():
    arr = [random.randint(1,10000) for _ in range(1000)]
    merge_sort(arr)

mem = memory_usage(test_memory)
print("Memory Usage:", mem)
```

Algorithm Comparison Table

Algorithm	Paradigm	Time Complexity	Use Case
Merge Sort	Divide & Conquer	O(n log n)	Large datasets
Quick Sort	Divide & Conquer	Avg O(n log n)	Fast sorting
Activity Selection	Greedy	O(n log n)	Scheduling
Knapsack	Dynamic Programming	O(nW)	Optimization

Algorithm	Paradigm	Time Complexity	Use Case
TSP	Brute Force	O(n!)	Path optimization

Reflection & Discussion

Trade-offs

- Greedy: Fast but not always optimal
- Dynamic Programming: Optimal but memory intensive
- Divide & Conquer: Balanced approach

Real-world Applications

- Logistics -> TSP
- Scheduling -> Greedy
- Finance -> DP

Recursion Depth

- Recursive algorithms may cause stack overflow for large inputs

---

References

- CLRS (Introduction to Algorithms)
- MIT OpenCourseWare
- Python Docs (matplotlib, memory_profiler)